

GradCafé Pipeline Limitations & Architectural Reflection

Module 3 Analysis Deliverable | Data Engineering, Schema Constraints, and Scraping Vulnerabilities

1. Web Scraping & Ingestion Brittleness

Relying on HTML scraping from platforms like The Grad Café introduces severe structural vulnerabilities. Any front-end template redesign, dynamic DOM manipulation via JavaScript, or rate-limiting changes directly breaks parser selectors. In Module 2, unstructured string formatting led to significant field blending—such as degrees being concatenated directly into program titles ('Physics PhD'). Without strict upstream structural guarantees, scrapers require continuous heuristic regular expressions to clean incoming records.

2. Database Constraints & Entity Disambiguation

A single-table relational schema ('applicants') enforces atomicity at the row level but fails to normalize institutional and academic relationships. Universities, degree levels, and programs should be separated into relational entities linked via foreign keys. Because admissions entries are user-submitted, variations like 'MIT', 'Mass Tech', and 'Massachusetts Institute of Technology' represent identical institutions but evaluate differently in basic SQL ILIKE queries unless LLM-assisted normalization or canonical lookup tables are applied.

3. LLM Field Reliability vs. Deterministic Parsing

While zero-shot LLM prompts resolve complex, unstandardized text variations, they introduce non-deterministic latencies and minor hallucination risks. When normalizing 30,000+ records, LLM parsing costs and throughput constraints necessitate asynchronous batching or fallback rules. The LLM fields proved critical for disambiguating university names where standard regex patterns failed, but strict JSON schema validation is mandatory before writing predictions directly into relational tables.

4. Architectural Scalability & Background Workflows

Triggering heavy ingestion or statistical recalculations inside synchronous HTTP request-response cycles risks blocking the web server and dropping client connections. In our Flask implementation, lightweight threading isolates worker routines from the web interface. In a full production deployment, this architecture should be decoupled using dedicated task queues like Celery or Redis Queue (RQ) backed by an independent worker fleet, guaranteeing transactional retries and worker fault tolerance.